

DAY 10: MESSAGING & CAPSTONE

Data Bytes Technologies · Java 25 & Modern Stack

Two acts today. First, the app finally **speaks Kafka** in code: the same topic you drove by hand on Day 7. Then the **capstone**: we run the whole fortnight, end to end, and look at where to go next.

// WHERE WE ARE

- **Day 7:** you drove Kafka by **raw CLI**: `kafka-console-producer` / `-consumer` against the `applicant-registered` topic. You *were* the producer and the consumer.
- **Day 9:** the KYC Service got a **real database**: `register` validates and persists to PostgreSQL via Panache, behind a clean REST API.

So we have a persistent service, and a topic we know how to talk to **by hand**. Today the **app** speaks that same topic **in code** (producing and consuming `applicant-registered`) and then we run the whole platform.

// BY THE END OF TODAY, YOU CAN...

- explain why a **channel** is just a name your code sends to, mapped to a Kafka **topic** by config
- **publish** `applicant-registered` from the KYC Service via a `@Channel` emitter, keyed by applicant id
- **consume** it in a Screening Service with an `@Incoming` method: no HTTP between them
- run the **whole platform** with one command and follow one applicant REST → Postgres → Kafka → background check
- name the **next steps** (native images, security, Kubernetes) and why each fits what you built

// ACT 1: THE APP SPEAKS KAFKA

The last piece of new code: the KYC Service **announces** what it did, and a second service **reacts**, without the two ever calling each other.

// A CALL VERSUS AN EVENT: THE DAY 7 IDEA, IN CODE

`register` already persists. What if it should *also* run a background check, update analytics, warm a cache?

- **A call:** the KYC Service phones each one and waits. If one is slow or down, the applicant is too.
- **An event:** the KYC Service says "an applicant registered" *once*, and moves on. Whoever cares listens.

💡 **In plain terms:** an event records that **a fact happened**. The producer doesn't know or care who reacts. That's the decoupling you saw on Day 7, now built into the app.

// FROM RAW CLI TO A DECLARATIVE CHANNEL

On Day 7 you ran the producer and consumer yourself. In Quarkus you don't touch the Kafka client at all. You talk to a **channel**.

💡 **In plain terms:** a **channel** is just a *name your code sends to or receives from*. `application.properties` decides that the channel `applicant-registered` is really the Kafka **topic** `applicant-registered`. Swap the wiring, keep the code.

- a `@Channel("applicant-registered")` **emitter** *sends* (the producer you played on Day 7)
- an `@Incoming("applicant-registered")` **method** *receives* (the consumer you played on Day 7)
- config maps each channel to a topic, a key/value **serializer**, and a consumer **group**

// ONE PICTURE: TWO SERVICES, ONE TOPIC



The two services share **one name**. Neither imports the other. The topic is the contract.

// THE CONFIG THAT WIRES CHANNEL → TOPIC

The code never names Kafka. `application.properties` does, separately for each side:

```
# KYC Service - outgoing
mp.messaging.outgoing.applicant-registered.connector=smallrye-kafka
mp.messaging.outgoing.applicant-registered.topic=applicant-registered
mp.messaging.outgoing.applicant-registered.value.serializer=org.apache.kafka.common.serialization.StringSerializer

# Screening Service - incoming
mp.messaging.incoming.applicant-registered.connector=smallrye-kafka
mp.messaging.incoming.applicant-registered.topic=applicant-registered
mp.messaging.incoming.applicant-registered.group.id=screening-service
mp.messaging.incoming.applicant-registered.value.deserializer=org.apache.kafka.common.serialization.StringDeserializer
mp.messaging.incoming.applicant-registered.auto.offset.reset=earliest
```

`group.id` and `auto.offset.reset=earliest` are the **consumer group** and **offset** ideas from Day 7, now two lines of config instead of CLI flags.

// THE EVENT ITSELF IS A RECORD

What travels on the wire? A plain Java **record**, serialised to JSON:

```
public record ApplicantRegisteredEvent(  
    Long applicantId, String fullName, String nationalId, String country,  
    String email, String status, Instant occurredAt,  
    String correlationId) {}
```

💡 **In plain terms:** the event is a **fact about the past**: "this applicant registered, here are its details." Named in the past tense, immutable, self-contained.

// JSON NOW, AVRO LATER

💡 **In plain terms:** we send **JSON strings**: readable, zero ceremony, easy to eyeball in kafka-ui. Perfect for learning and for plenty of real systems.

- **At scale**, a binary format like **Avro** with a **Schema Registry** buys compact messages and **enforced, evolvable schemas**: a producer literally cannot ship a change that breaks consumers.
- The cost is tooling and a **registry to run**. A deliberate trade: JSON is the right call today.

// 🖥️ CODE ALONG: REGISTER RECORDS THE EVENT

🖥️ Code along: Day 9's `register` only persisted. Watch it also record the fact that an applicant registered.

```
// AFTER (checkpoint/day-10): persist, then record applicant-registered
@Transactional
public Applicant register(Applicant draft) {
    draft.status = ApplicantStatus.PENDING;
    applicants.persist(draft);           // draft.id is set after this

    ApplicantRegisteredEvent event = new ApplicantRegisteredEvent(
        draft.id, draft.fullName, draft.nationalId, draft.country,
        draft.email, draft.status.name(),
        Instant.now(), currentCorrelationId());
    recordForPublishing(event, draft.id); // ← announce the fact
    return draft;
}
```

// 🖨️ CODE ALONG: THE @CHANNEL EMITTER

🖨️ **Code along:** inject an emitter on the `applicant-registered` channel and send the event **keyed by applicant id**.

```
@Inject
ApplicantService(/* ... */ @Channel("applicant-registered") MutinyEmitter<String> emitter) { /* ... */ }

String value = mapper.writeValueAsString(event); // the record → JSON string

Message<String> message = Message.of(value).addMetadata(
    OutgoingKafkaRecordMetadata.<String>builder()
        .withKey(String.valueOf(applicantId)) // key = applicant id → same partition
        .build());
emitter.sendMessage(message).await().indefinitely();
```

`@Channel("applicant-registered")` is the **outgoing** side. `.withKey(applicantId)` is the **partition key** from Day 7: same applicant, same partition, order preserved.

// 🖥️ CODE ALONG: THE @INCOMING CONSUMER

🖥️ **Code along: the Screening Service:** a method on the `@Incoming` channel, fed each message by Quarkus.

```
@Incoming("applicant-registered")
@Blocking
public CompletionStage<Void> consume(Message<String> message) {
    ApplicantRegisteredEvent event = mapper.readValue(message.getPayload(), ApplicantRegisteredEvent.class);
    Screening screening = backgroundCheck.run(event);    // watchlist + risk score
    store.record(screening);
    LOG.info("Screened applicant %d -> %s"
        .formatted(event.applicantId(), screening.decision().name()));
    return message.ack();                // handled -> advance the offset
}
```

No HTTP, no call to the KYC Service. Quarkus delivers each message; we record a screening and **ack** it. That's the whole consumer.

// TEST IT WITHOUT A BROKER

Because the code only ever names a **channel**, tests can point that channel somewhere else. We swap Kafka for an **in-memory** connector under the `%test` profile:

```
# KYC Service test profile
%test.mp.messaging.outgoing.applicant-registered.connector=smallrye-in-memory
# Screening Service test profile
%test.mp.messaging.incoming.applicant-registered.connector=smallrye-in-memory
```

So `./mvnw -B verify` asserts the producer emitted and the consumer reacted: **no broker, no Docker** for that part. Postgres still runs for real via Dev Services. Same code, different wiring.

// TRY IT NOW (5 MIN): WATCH THE EVENT TRAVEL

Bring up the platform, post an applicant, and look in **two** places:

```
docker compose -f infra/compose.yaml --profile apps --profile observability up -d --build

curl -s localhost:8080/applicants -H 'content-type: application/json' \
  -d '{"fullName":"Alice Tan","nationalId":"S1234567A","dateOfBirth":"1990-05-01","country":"SG","email":"alice@e
```

- **kafka-ui** at <http://localhost:8081> → topic `applicant-registered` → your message is there
- the **Screening Service**: `docker compose -f infra/compose.yaml logs screening-service` (each `Screened applicant ...`)

// A REAL-WORLD NOTE: THE SAVE-THEN-PUBLISH SEAM

`register` does **two writes**: to the **database**, then to the **broker**. They are **not** one atomic action.



💡 **Awareness, not depth:** the production fix is the **transactional outbox**: write the event as a DB row in the *same* transaction, then a poller relays it to Kafka. The reference app does exactly this. Delivery becomes **at-least-once**, so the consumer **dedupes** (idempotency).

// 🖐️ EVERYONE WITH ME?

You should, right now, be able to:

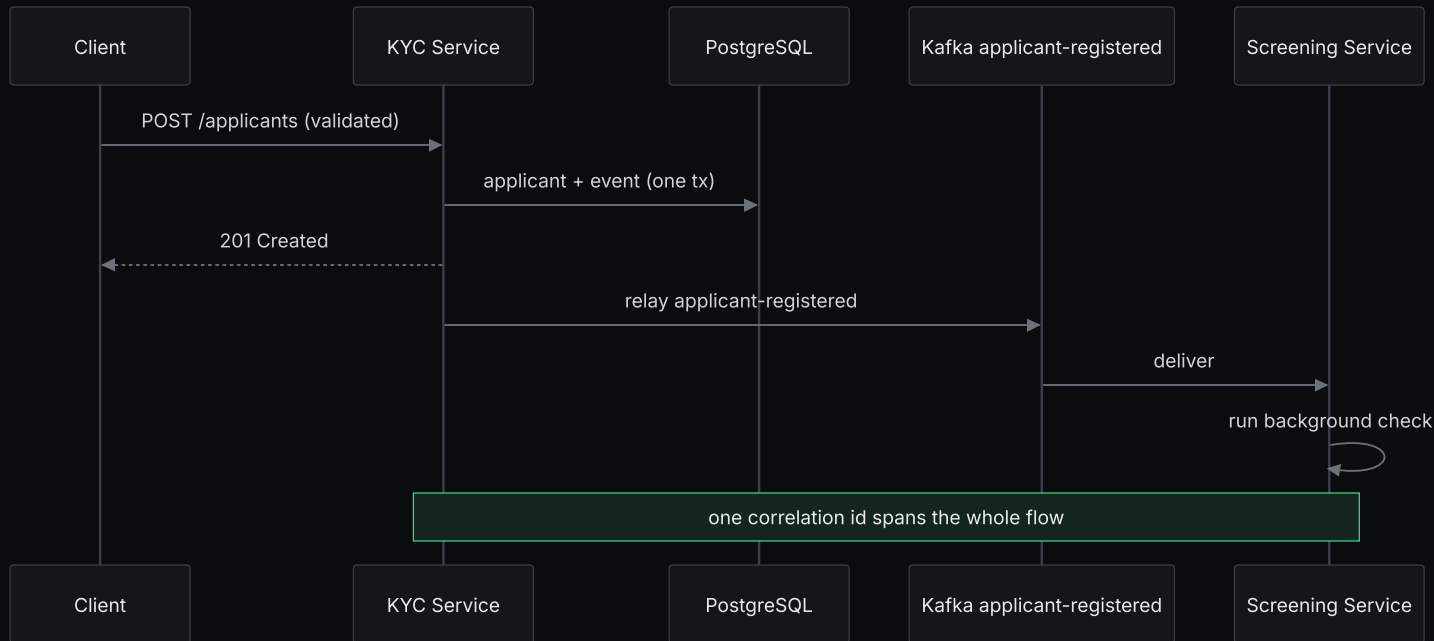
- POST an applicant and get **201** back
- **see the message** in kafka-ui on the `applicant-registered` topic
- **see a screening** appear in the Screening Service

If any of those is missing, flag it now. The capstone builds directly on this end-to-end flow.

// ACT 2: THE CAPSTONE

The system is **done**. Now we run the whole fortnight end to end, follow one applicant across every hop, and look outward at what's next.

// WHAT YOU BUILT: ONE SYSTEM, END TO END



// THE TAKEAWAY OF THE WHOLE COURSE

💡 **In plain terms:** every individual piece was small and testable. The hard, valuable part is that they **compose into a system** whose failure modes you can actually reason about.

The architecture model lives in `architecture/workspace.dsl`; render it with Structurizr Lite. The runtime topology poster is regenerated from `diagrams/` with `uv run python -m src.kyc_platform`.

// TRY IT NOW (5 MIN): RUN THE WHOLE PLATFORM

From the repo root, bring up every service and its infrastructure:

```
docker compose -f infra/compose.yaml --profile apps --profile observability up -d --build
```

Then create one applicant and watch it travel:

```
curl -s localhost:8080/applicants -H 'content-type: application/json' \  
  -d '{"fullName":"Alice Tan","nationalId":"S1234567A","dateOfBirth":"1990-05-01","country":"SG","email":"alice@e
```

You should get `201 Created`. Within a moment the **Screening Service** shows it consumed the event. The whole system just did its job.

// FOLLOW ONE APPLICANT ACROSS EVERY HOP

The applicant touched **four components**. You can follow it as a single story:

- the same **correlation id** rides in the event body, so it appears in the KYC Service log **and** the Screening Service log: `grep one id, see every hop`
- the **applicant id** is the Kafka **key**, so kafka-ui shows you exactly which partition it landed on
- `GET /screenings` proves the consumer reacted, with **no call** back to the KYC Service

💡 **In plain terms:** without one shared id, a multi-service bug is several separate haystacks. With it, there's one thread you can pull from the REST call all the way to the screening.

// 🖐️ EVERYONE WITH ME?

You should have, right now:

- the **full stack** up via `docker compose ... up -d --build`
- a `201` from a `POST /applicants`
- the **Screening Service** showing it consumed the event

If any of those isn't true, flag it now. The rest of today is a tour, but this demo is the thing you'll want working when you show someone what you built.

// THE FORTNIGHT, WHOLE: WEEK 1: YOUR TOOLBOX

Every day was one tool. Here's how they stack:

- **Day 1: Git:** history you can read, branch, and recover
- **Day 2: Docker:** a reproducible environment for everyone
- **Day 3: Java for polyglots:** records, sealed types, pattern matching
- **Day 4: Concurrency I:** threads, the memory model, what goes wrong
- **Day 5: Concurrency II:** virtual threads, structured concurrency

💡 Week 1 was **skills**: the language and the habits the app is built with.

// THE FORTNIGHT, WHOLE: WEEK 2: THE RUNNING SYSTEM

- **Day 6: PostgreSQL:** talk to it with `psql`; schema and SQL
- **Day 7: Kafka:** drive topics by CLI; topic, partition, group, offset
- **Day 8: Quarkus fundamentals:** REST, validation, config, tests
- **Day 9: Quarkus + persistence:** Panache, Flyway, Dev Services
- **Day 10: Quarkus + messaging:** `@Channel` / `@Incoming`, the capstone

💡 Week 2 you **built**: each tool from Week 1 became part of one Quarkus service, commit by commit.

// NATIVE IMAGES: START IN MILLISECONDS

Quarkus can compile a service to a **native executable** with GraalVM/Mandrel. It boots in **milliseconds** and uses a fraction of the memory.

```
./mvnw -B package -Dnative
```

💡 **In plain terms:** instead of shipping bytecode that a JVM runs, you ship a real machine binary with no JVM at all. That's why it starts almost instantly.

// NATIVE IMAGES: PICK PER WORKLOAD

- **Closed-world build:** the binary can't discover code at runtime, so reflection and resources need config (most extensions ship it for you). The build is **slow and memory-hungry**.
- **No JIT:** no runtime optimiser, so peak **throughput** on long-running hot services can trail the JVM.
- **Native:** fast-scaling, short-lived, memory-constrained: serverless, high-density pods, CLIs.
- **JVM:** steady, throughput-bound services that run hot for hours and benefit from the JIT.

It's the **same code** and the same **Quarkus REST** endpoints. Only the packaging changes. Choose by deployment shape, not by habit.

// TRY IT NOW (OPTIONAL, SLOW): BUILD NATIVE

If you have the toolchain (GraalVM/Mandrel, or a container build), from the KYC Service:

```
./mvnw -B package -Dnative          # this takes minutes, that's the cost
./target/kyc-service-*-runner       # then watch the boot line
```

Compare the **start-up time** and **resident memory** against `./mvnw quarkus:dev` from earlier in the week.

Don't block the room on this. Kick it off and let it bake while we talk roadmap. The numbers are the lesson: milliseconds to boot, a fraction of the RAM.

// WHAT WE KEPT DELIBERATELY SIMPLE

A real platform has more, and we left it *light on purpose*, so the core stayed clear:

- **the outbox**: we named the save-then-publish seam and the fix; the reference app implements it
- **observability**: one **correlation id** end to end is enough to follow a request; metrics, logs, and distributed tracing are the full picture
- **security**: endpoints are open here so we could focus on data and event flow

💡 **In plain terms**: these aren't gaps, they're **next layers**. You now have the spine they bolt onto, and the vocabulary to learn them.

// SELF-STUDY ROADMAP

You have the foundation; here's where to go next, and **why each fits** what you built:

- **Kubernetes:** the natural home for these containers; your **health checks** and **config profiles** map straight onto probes and ConfigMaps.
- **Security (OIDC + JWT):** put `quarkus-oidc` in front; authorisation becomes annotations (`@RolesAllowed` , `@Authenticated`) and the framework verifies the token.
- **CI/CD:** the repo already has a CI workflow; extend it to **build and publish images** and run the **native build** on a tag.
- **gRPC & performance testing:** low-latency service-to-service calls, and load-testing the platform to tune it against real traffic.

// COMMON TRAPS

- **Treating a topic like a queue:** reading doesn't delete. Many consumers, replayable history (Day 7).
- **Expecting global ordering:** order holds only **within a partition**. The key keeps an applicant's events together.
- **Forgetting to `ack`:** the offset won't advance, and the message reappears on restart.
- **Sending inside the request and calling it done:** that's the save-then-publish seam; the outbox is the production fix.

// RECAP, IN PLAIN TERMS

Today the platform became a **system of cooperating services**, and you ran the whole fortnight:

- a **channel** is a name your code sends to; config maps it to a Kafka **topic**
- the KYC Service **publishes** `applicant-registered` keyed by applicant id; the Screening Service **reacts** via `@Incoming`, no direct call
- one command (`docker compose ... up -d --build`) runs REST → Postgres → Kafka → background check, end to end
- the save-then-publish **seam** is real; the **outbox** + idempotency are the production fix
- **native vs JVM** is a packaging choice; pick by deployment shape

// TODAY'S LAB: THE CAPSTONE

 **Capstone:** wire the KYC Service to Kafka via Quarkus messaging, bring up the full stack, and **demonstrate the end-to-end flow**, then close any gaps you find.

Brief: `labs/day-10/` · Start from `git checkout checkpoint/day-09` · Solution: `checkpoint/day-10`

Acceptance: `./mvnw -B verify` green for both services, `docker compose ... up -d --build` runs the platform, a `POST /applicants` returns `201`, the event reaches the topic, and the Screening Service consumes it.

// 🎉 TAKE A MOMENT

Two weeks ago this was loops and `null` checks. Now:

- you stood up a **validated REST service**, gave it a **real database**, and made it **publish events**
- a second service **consumes them**, and you can **follow one applicant** across the whole platform
- you know **when to ship it native**, and **where to grow it next**

That is a genuine, modern, production-shaped system. **Well done.** 🎉

// TAKEAWAY

You built a complete KYC platform on **Java 25** and **Quarkus** (validate, persist, publish, consume) where every service stayed small enough to reason about, and the **topic is the contract** that lets them grow apart.

The pieces stayed simple; you can reason about the whole. That's modern software engineering. Thank you, and keep building.